

Artificial Intelligence Concepts and Machine Learning Techniques (AI – 103)

Walaa H. Elashmawi
Professor of AI

 walaa.hassan@miuegypt.edu.eg

Model Evaluation, Cross-Validation & Hyperparameter Tuning

Error

- Errors represent the difference between predicted and actual values and are used to evaluate a model's performance through **loss (error) functions**.
- A **smaller error value** indicates a better-performing model, though small errors can sometimes be acceptable if they have minimal impact on results.
- The **acceptable (negligible) error threshold** varies by application — for instance, a minor error in **house price prediction** might be acceptable, while the same magnitude could be critical in **medical diagnosis**.



- ☐ Measures how well a model generalizes to unseen data.
- ☐ Helps detect overfitting and underfitting.
- ☐ Key metrics: Accuracy, Precision, Recall, F1-score, ROC-AUC, RMSE,

Mean Squared Error (MSE)

- The **Mean Squared Error (MSE)** measures the average of the squared differences between the actual and predicted values for all data points.
- It quantifies how close predictions are to the true values and is expressed as:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y^{(i)} - \hat{y}^{(i)})^2$$

Where:

- $n \rightarrow$ total number of data points
 - $y(i) \rightarrow$ predicted value
 - $\hat{y}(i) \rightarrow$ actual value
- MSE is also known as the average squared error. It is highly sensitive to outliers, as large errors are squared and contribute disproportionately to the overall score.

Mean Absolute Error (MAE)

- The **Mean Absolute Error (MAE)** represents the average of the absolute differences between the actual and predicted values across all data points.
- Unlike MSE, which squares the errors, MAE uses **absolute values**, making it **less sensitive to outliers**.

$$MAE = \frac{1}{n} \sum_{i=1}^n |y^{(i)} - \hat{y}^{(i)}|$$

Where:

- $n \rightarrow$ total number of data points
 - $y(i) \rightarrow$ actual value
 - $\hat{y}(i) \rightarrow$ predicted value
- MAE provides a clear interpretation of the **average magnitude of errors** in the model's predictions, regardless of their direction (positive or negative).

Mean Absolute Percentage Error (MAPE)

- The **Mean Absolute Percentage Error (MAPE)** measures the **average percentage difference** between actual and predicted values.
- It is computed by first finding the ratio of the absolute error to the actual value for each data point, then averaging these ratios and multiplying by 100.

$$MAPE = \frac{100}{n} \sum_{i=1}^n \left| \frac{y^{(i)} - \hat{y}^{(i)}}{\hat{y}^{(i)}} \right|$$

Where:

- $n \rightarrow$ total number of data points
- $y(i) \rightarrow$ actual value
- $\hat{y}(i) \rightarrow$ predicted value
- MAPE (also called **Mean Absolute Percentage Deviation, MAPD**) expresses error as a **percentage**, making it easy to interpret.
- It is **robust to outliers**, like MAE, and is preferred over MSE or RMSE when the dataset contains extreme values.

Mean Absolute Percentage Error (MAPE)

- The **Huber Loss Function**, introduced by **Peter Jost Huber (1964)**, is primarily used for **regression problems**.
- It is designed to be **less sensitive to outliers** compared to the Mean Squared Error (MSE) by combining the properties of both the **squared error** and **absolute error** functions in a **piecewise format**:

$$L = \begin{cases} \frac{1}{2}(y^{(i)} - \hat{y}^{(i)})^2, & \text{if } |y^{(i)} - \hat{y}^{(i)}| \leq \delta \\ \delta(|y^{(i)} - \hat{y}^{(i)}| - \frac{1}{2}\delta), & \text{otherwise} \end{cases}$$

Where:

δ (delta) is a small positive threshold value.

Key Characteristics:

- Acts **quadratic (like MSE)** for small errors.
- Acts **linear (like MAE)** for large errors.
- Ensures smooth transition at $|y^{(i)} - \hat{y}^{(i)}| = \delta$.
- Provides **robustness** to outliers while maintaining sensitivity to small errors.

Example:

Given:

Predicted values ($\hat{y}$): [0.7, 1.1, 1.5, 1.9, 2.3, 2.7, 3.1]

Actual / true values (y): [1.08, 1.2, 1.4, 2.1, 1.9, 7, 2.9]

Number of data points (n) = 7, $\delta = 1.35$

1. Mean Squared Error (MSE)

$$\text{MSE} = [(1.08 - 0.7)^2 + (1.2 - 1.1)^2 + (1.4 - 1.5)^2 + (2.1 - 1.9)^2 + (1.9 - 2.3)^2 + (7 - 2.7)^2 + (2.9 - 3.1)^2] / 7$$

$$\text{MSE} = 18.894 / 7 = \mathbf{2.699}$$

2. Mean Absolute Error (MAE)

$$\text{MAE} = [|1.08 - 0.7| + |1.2 - 1.1| + |1.4 - 1.5| + |2.1 - 1.9| + |1.9 - 2.3| + |7 - 2.7| + |2.9 - 3.1|] / 7$$

$$\text{MAE} = 5.677 / 7 = \mathbf{0.811}$$

3. Huber Loss

$$|7 - 2.7| = 4.3 > 1.35 \rightarrow \text{outlier}$$

$$L_6 = \delta(|y_6 - \hat{y}_6| - \frac{1}{2}\delta) = 1.35 \times (4.3 - 0.675) = 4.894$$

$$\text{Huber Loss} = [0.5 \times ((1.08 - 0.7)^2 + (1.2 - 1.1)^2 + (1.4 - 1.5)^2 + (2.1 - 1.9)^2 + (1.9 - 2.3)^2 + (2.9 - 3.1)^2) + 4.894] / 7$$

$$\text{Huber Loss} = (0.2022 + 4.894) / 7 = \mathbf{0.728}$$

Final Results

Metric	Value
MSE	2.699
MAE	0.811
Huber Loss	0.728

Cross Entropy

- In **machine learning**, cross-entropy is used as a **loss function for classification tasks**, measuring the difference between **true labels** and **predicted probabilities**.
- The **cross-entropy loss** is given by:

$$L_{CE} = - \sum_{i=1}^n y_i \log(\hat{y}_i)$$

For **binary classification (n = 2)**, it simplifies to:

$$L_{CE} = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

- Cross-entropy loss quantifies how well predicted probabilities ($\hat{y}$) match the true labels (y).
- It is minimized during training to improve model accuracy.
- **Lower cross-entropy loss → better model performance.**
- Commonly used in models like **Logistic Regression, Softmax Classifiers, and Neural Networks.**

Example:

Consider: three-class classification model for a specific input

Given: Predicted values $\hat{y} = [0.02, 0.97835, 0.00165]$

Actual / true values (y): $y = [0, 1, 0]$

Determine the log loss or cross-entropy loss

$$\begin{aligned}\text{Cross entropy loss (L)} &= -\sum y_i \log(\hat{y}_i) \\ &= -(0 \times \log(0.02) + 1 \times \log(0.97835) + 0 \times \log(0.00165)) \\ &= -\log(0.97835) \\ &= 0.0095.\end{aligned}$$

So, the cross-entropy loss is 0.0095.

Confusion Matrix

		Predicted class		
		Positive	Negative	
Actual class	Positive	True Positive (TP)	False Negative (FN) Type II error	Sensitivity $\frac{TP}{(TP+FN)}$
	Negative	False Positive (FP) Type I error	True Negative (TN)	Specificity $\frac{TN}{(TN+FP)}$
		Precision $\frac{TP}{(TP+FP)}$	Negative Predictive Value $\frac{TN}{(TN+FN)}$	Accuracy $\frac{TP+TN}{(TP+TN+FP+FN)}$

Example:

Let us assume that “Cat” is the positive class and “Dog” is the negative class. So:

- ✓ True positive, T P = is cat and is classified as cat = 48.
- ✓ True negative, T N = is not cat and is not classified as cat = is dog and is classified as dog = 30.
- ✓ False positive, FP = is not cat but is classified as cat = $50 - 30 = 20$.
- ✓ False negative, F N = is cat but is classified as dog = 2.

$$\text{Accuracy} = (48 + 30) / (48 + 30 + 20 + 2) = 78 / 100 = 0.78 \approx 78\%$$

78% of predictions were correct

$$\text{Precision} = 48 / (48 + 20) = 48 / 68 = 0.706 \approx 70.6\%$$

70.6% of positive predictions were correct

$$\text{Recall} = 48 / (48 + 2) = 48 / 50 = 0.96 = 96\%$$

Model detected 96% of actual positives

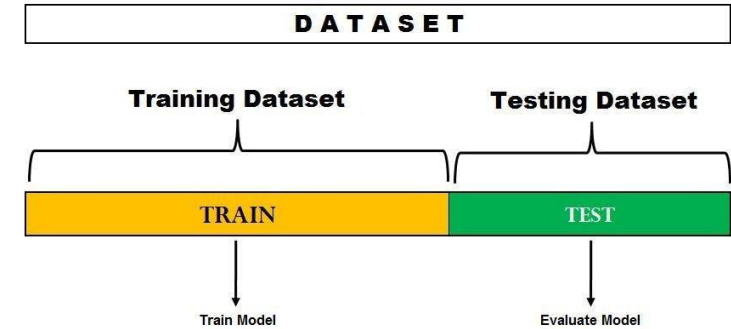
$$\text{F1} = 2 \times (0.706 \times 0.96) / (0.706 + 0.96) = 0.814$$

Balanced metric between precision & recall

The model seems to perform better for cat images than dog images. For this reason, the precision score of this model is somewhat lower, even though it has a higher recall score.

Hold-Out Method

- Split dataset into Train (70%) and Test (30%) OR Train (80%) and Test (20%).
- Train model on training set, evaluate on test set.
 - Pros: Simple and quick.
 - Cons: Results vary with random splits.

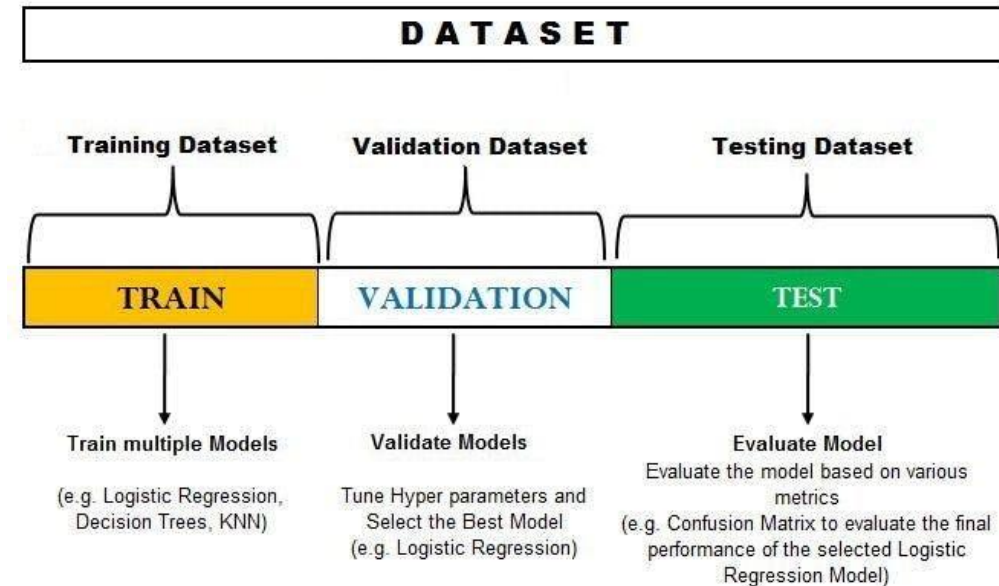


Cross-Validation

This is a statistical method to evaluate the performance of a model. It is a much more stable way to evaluate data than based on training and testing datasets

It is also an effective way to prevent overfitting. If the model is trained with a larger dataset, the probability of overfitting lessens.

- Reduces variance and bias.



Most Common Types of Cross- Validation

K-Fold: Common method for general tasks.

Stratified K-Fold: Maintains class balance.

Leave-One-Out (LOOCV): For small datasets.

Repeated K-Fold: Improves stability.



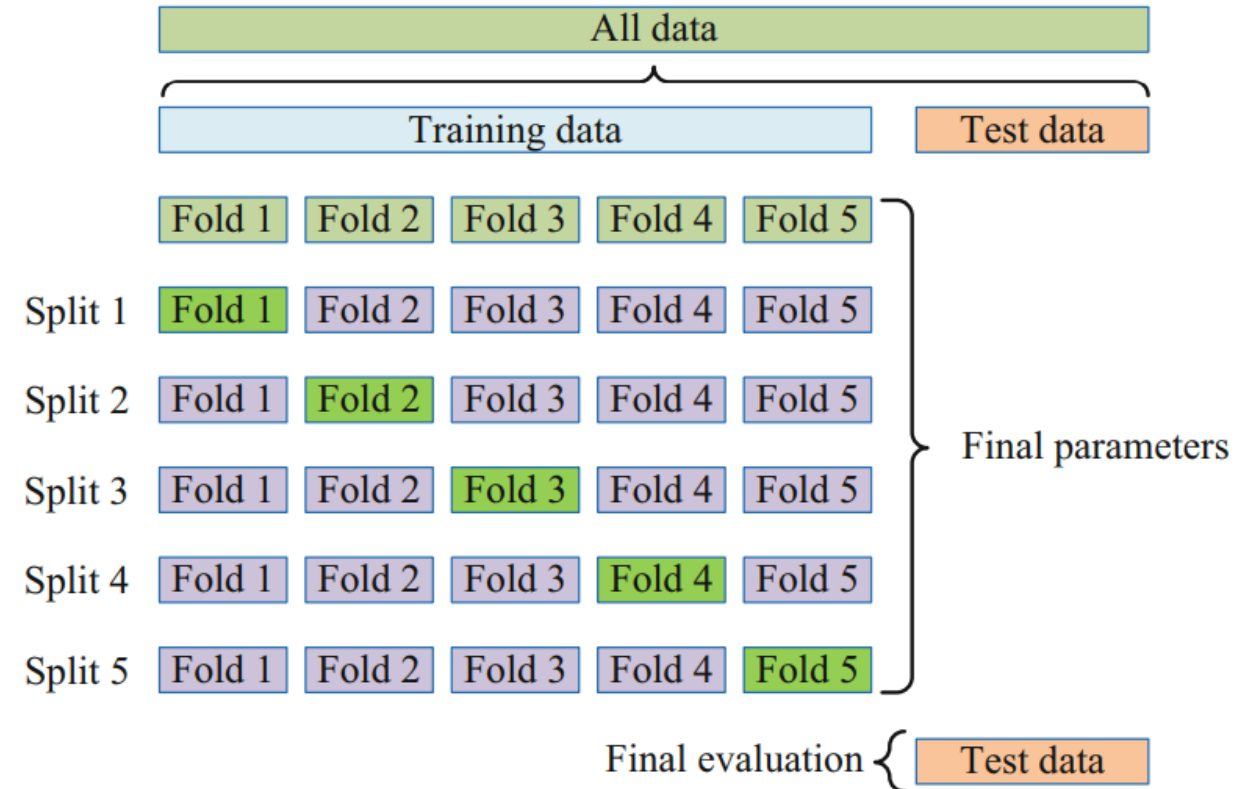
K-Fold Cross-Validation

Problem with Conventional Split:

- Typically, data is split as **75% training** and **25% testing**.
- This can be **unstable** when data is limited or when the model faces **unseen patterns**.

How K-Fold Works:

- Dataset is divided into K equal parts (folds).
- Model trains on K-1 folds and tests on the remaining fold.
- Process repeats K times and results are averaged (i.e., After all folds are tested, the **average of all errors** gives the model's **overall error rate** (or performance score).



Example: For K=5, model trains on 4 folds and tests on the remaining 1 — repeated 5 times.

Fivefold cross-validation visualization

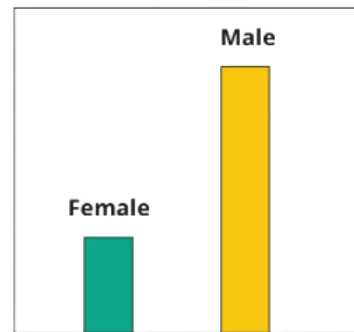
It Provides a **more stable and reliable estimate** of model accuracy compared to a single train-test split.

Stratified K-Fold Cross-Validation

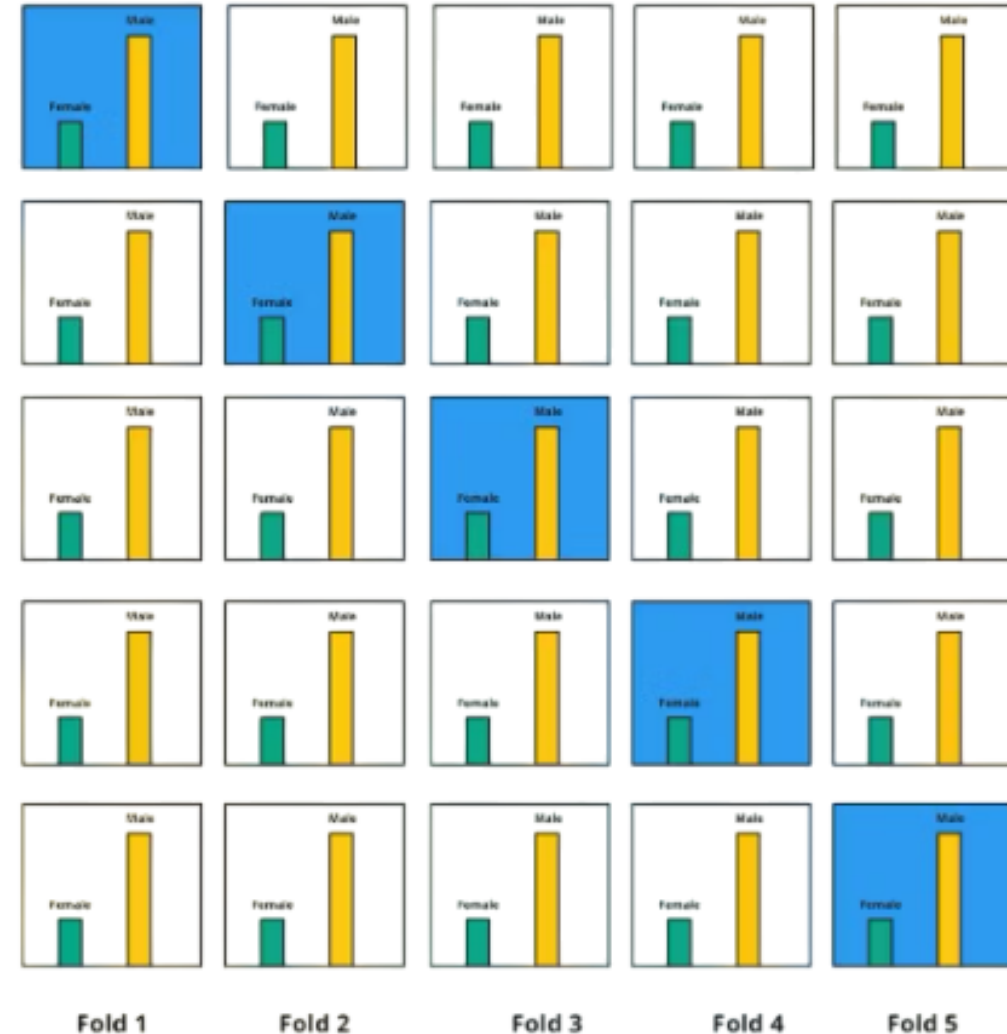
- Enhanced version of K-fold which used for imbalanced datasets.
- Each fold preserves the percentage of samples for each class (i.e., splits the dataset into k equal folds, each fold has the same ratio of instances of target variables that are in the complete dataset.)
- Ensures fair class representation in each fold.

Example: If dataset has 70% Class A and 30% Class B, each fold keeps this ratio.

This enables it to work perfectly for imbalanced datasets, but not for time-series data.



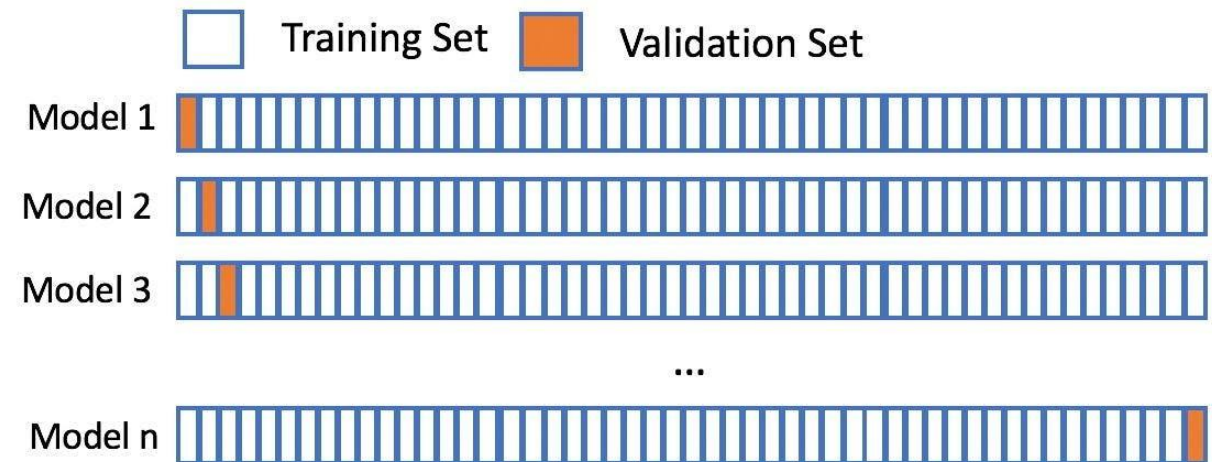
Target Class
Distribution



Leave-One-Out Cross-Validation (LOOCV)

- Each data sample is used once as a validation set.
- Model trains on all remaining samples (i.e., $n-1$ samples).

Example: For 100 samples, model trains on 99 and tests on 1 — repeated 100 times.



Very accurate but computationally expensive.

Repeated K-Fold Cross-Validation

- Runs K-Fold multiple times with different random splits

Example: 5-Fold CV repeated 3 times = 15 total evaluations.

Repeated K-Fold Cross Validation. A repeated 10-fold CV was applied. The 10-fold CV works by dividing the training data into 10 equal parts. These parts are iterated through 10 times. During each iteration, 9 of the 10 parts are treated as training data, and the remaining 10 th part as the validation set. The performance metrics are measured after each iteration.



Improves model reliability and reduces variance. Good for robust performance estimation.

Model Parameter and Hyperparameter

Aspect	Parameters	Hyperparameters
Definition	Internal variables learned automatically from the data during training.	External settings defined before training to control the learning process.
Learned From	Derived from data through optimization.	Set manually or tuned using validation methods.
Changes During Training	✅ Yes — values are updated continuously (e.g., via gradient descent).	❌ No — remain fixed during training.
Examples	• Regression coefficients • Weights and biases in neural networks • Cluster centroids	• Learning rate • Batch size • Kernel size • Number of epochs/iterations • Number of hidden layers • Choice of optimizer, loss function, or activation function
Role in Model	Represent what the model learns from the data.	Control how the model learns from the data.
Internal/External	Internal to the model.	External to the model.
Optimization	Found by training algorithms.	Chosen by trial, tuning, or search methods (e.g., grid search, random search).

Hyperparameter Space

Different ML algorithms come with different types of hyperparameters.

Polynomial Regression:

- Degree of polynomials

Logistic Regression:

- Regularization
- Class weights

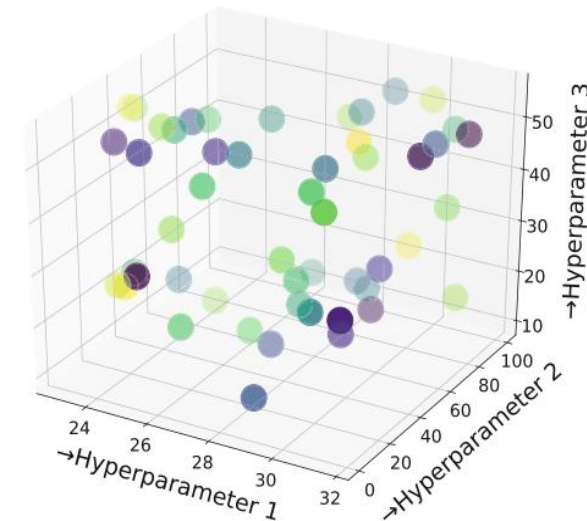
Neural Network:

- Number of layers
- Number of neurons
- Activation function
- Optimizer algorithm
- Learning rate

Decision Tree and Random Forest:

- Maximum tree depth
- Minimum split samples
- Minimum number of leaf samples
- Maximum number of features
- Split quality criterion

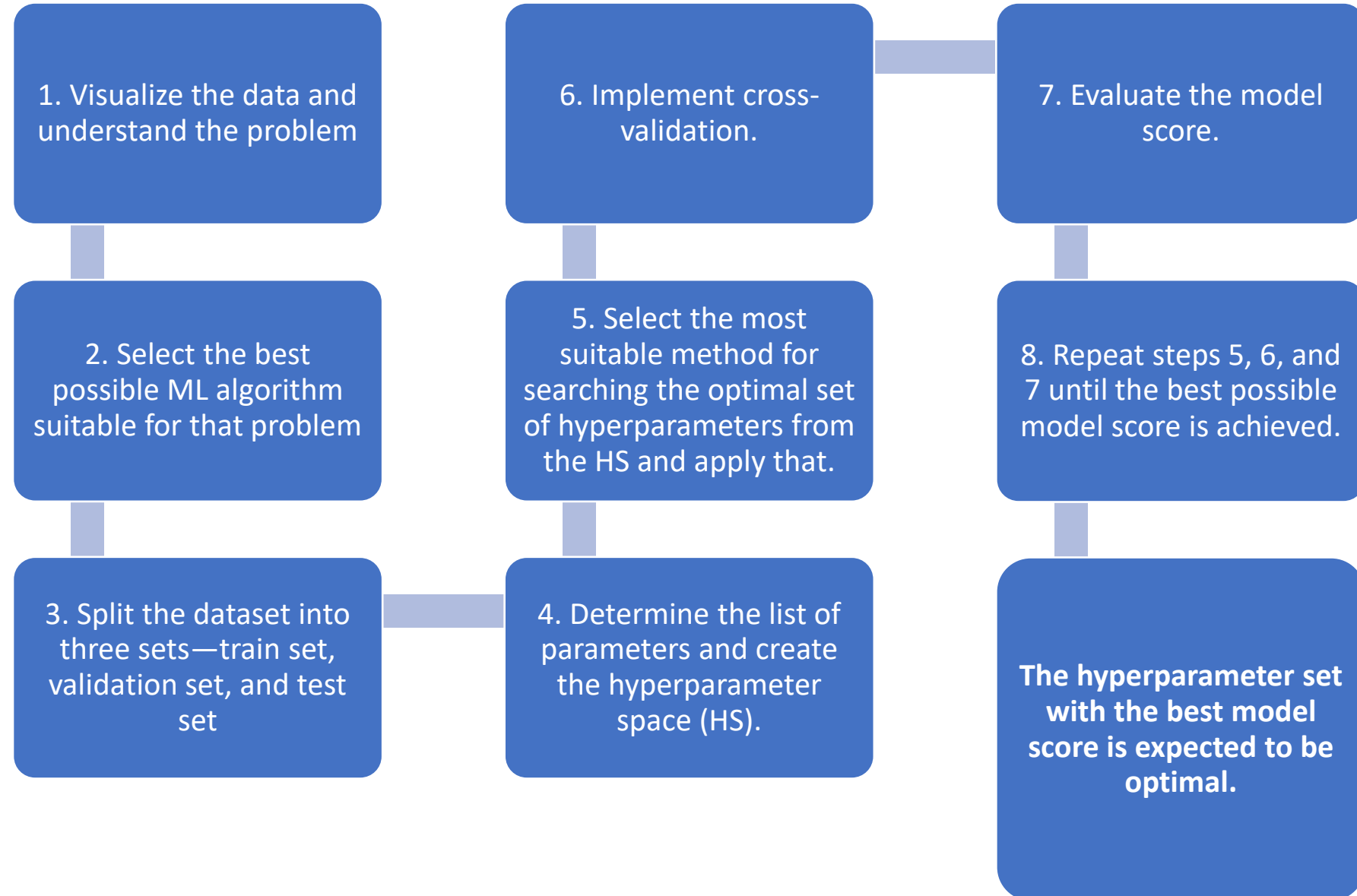
In general, the set of all possible solutions for a system is called the **search space**.



Hyperparameter Space in ML:

- Refers to **all possible combinations of hyperparameters** that can be used to train a model
- Each **axis** in this space represents one hyperparameter.
- Each **point** in this space corresponds to a unique set of hyperparameter values.
- The hyperparameter space is **n-dimensional**, where n = number of hyperparameters.
- To find the **optimal point** (best set of hyperparameters) through **hyperparameter tuning**.

The steps associated with hyperparameter tuning are given below:



Most Common Types of hyperparameter tuning

Manual Search

Exhaustive Grid Search

Halving Grid Search

Random Search

Halving Random Search

Bayesian Optimization

Gradient Based Optimization

Evolutionary Algorithm



Manual Search

Manual (Trial-and-Error) Hyperparameter Tuning

Definition: A basic hyperparameter tuning approach where parameters are chosen **manually** through trial and error.

Process:

1. A **hyperparameter space** is defined.
2. Different combinations are **tested manually** to find the best-performing set.

Advantages	Disadvantages
1. Simple and quick for experienced practitioners. 2. No extra computational cost.	1. Low chance of finding optimal parameters. 2. Not systematic and often inefficient for complex models.
Use Case:	Useful for small models or initial experimentation but not recommended for high-precision tasks.

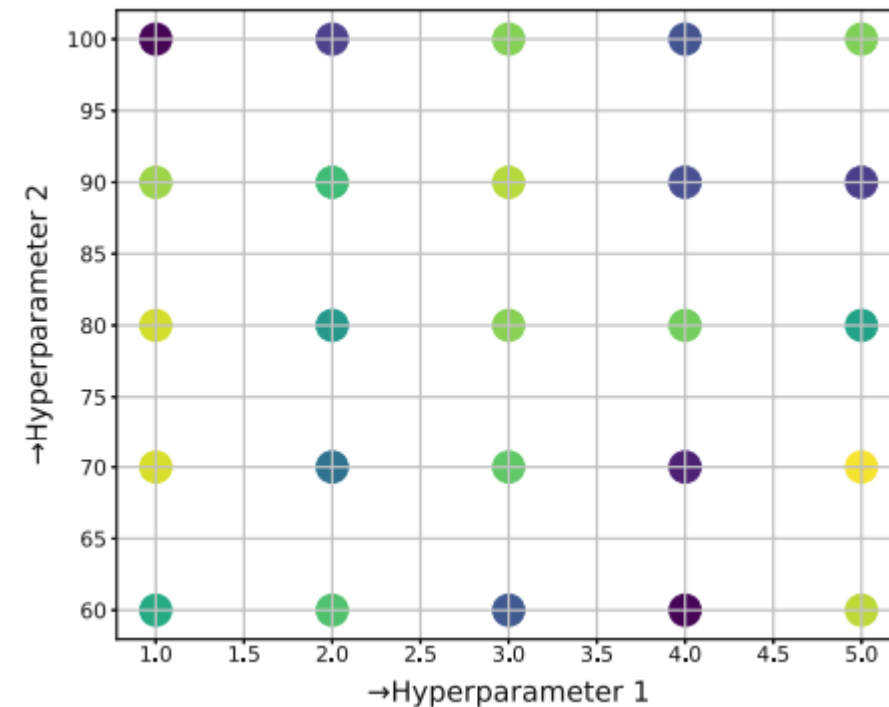
Exhaustive Grid Search

• **Definition:** An **exhaustive (brute-force)** method that tests **all possible combinations** of hyperparameters.

Steps:

1. Define a **hyperparameter space** (all possible values).
2. Perform **cross-validation** for each combination.
3. Select the combination with the **best evaluation metric**.

Advantages	Disadvantages
1. Comprehensive — ensures every combination is tested. 2. Can find the optimal hyperparameter set for the model.	1. Highly time-consuming and computationally expensive, especially for large datasets. 2. May lead to overfitting since the model is fine-tuned too closely to the validation data.
Use Case:	Suitable for small datasets or limited hyperparameter ranges , but not ideal for large-scale problems.



Exhaustive grid search in a two-dimensional hyperparameter space

Halving Grid Search

Definition: A faster and more efficient version of Grid Search that uses the **Successive Halving Algorithm (SHA)**.

Process:

1. Start by training models with **all hyperparameter combinations** using **limited resources** (short training time).
2. **Evaluate performance** and **eliminate half** of the weaker combinations.
3. **Increase computational resources** (e.g., training time or data) for the remaining combinations.
4. Repeat until **one best-performing set** of hyperparameters remains.

Advantages	Disadvantages
Much faster than standard Grid Search. Efficient resource allocation — focuses more on promising hyperparameter sets. Reduces unnecessary computation.	Might discard potentially good combinations early if they perform poorly in initial limited training.
Use Case:	Ideal when working with large datasets or complex models where full Grid Search is too slow.

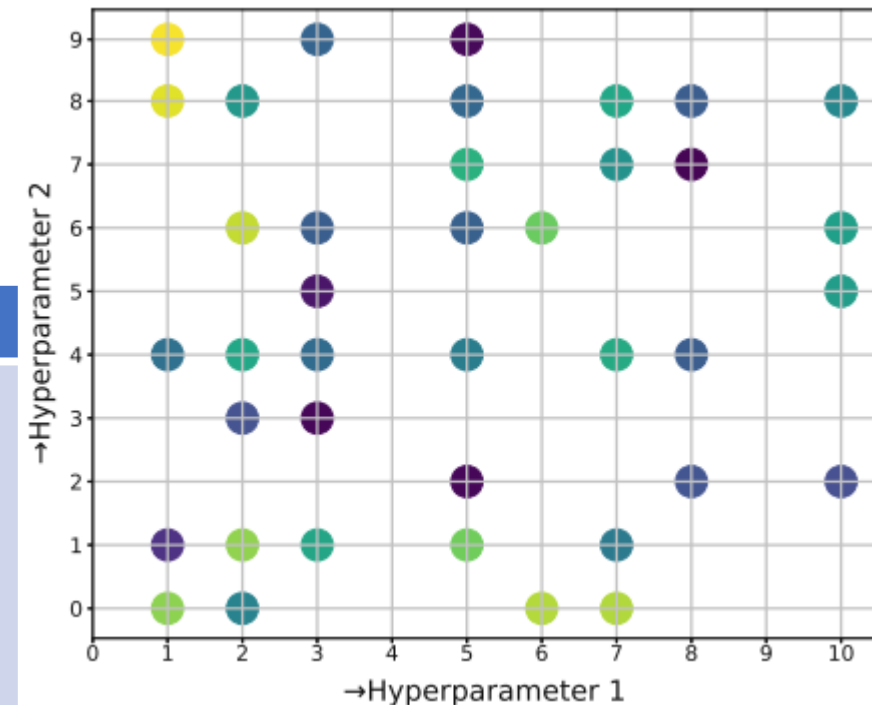
Random Search

•**Definition:** A **non-exhaustive tuning method** where hyperparameter combinations are **randomly selected** instead of testing all possible options.

Steps:

1. Randomly **sample combinations** from the hyperparameter space.
2. Perform **cross-validation** for each selected combination.
3. Identify the combination yielding the **best performance**.

Advantages	Disadvantages
1. Faster and computationally cheaper than Grid Search. 2. Reduces overfitting risk compared to exhaustive methods. 3. Effective when only a few hyperparameters significantly affect performance.	1. May miss the optimal combination because selection is random. 2. Can still be time-consuming with large datasets or many parameters.
Use Case:	Suitable for large hyperparameter spaces where Grid Search is impractical.



Random search in a two-dimensional hyperparameter space

Halving Random Search

Definition: An **efficient version of Random Search** that applies the **Successive Halving Algorithm (SHA)** to speed up the tuning process.

Process:

1. **Randomly select** hyperparameter combinations from the search space.
2. **Train models with limited resources** (e.g., few epochs or small data subsets).
3. **Eliminate the worst-performing half** of the combinations.
4. **Increase resources** for the remaining ones.
5. Repeat until the **best hyperparameter set** remains.

Advantages	Disadvantages
1. Faster and more efficient than standard Random or Grid Search. 2. Reduces computational cost by focusing resources on better-performing models. 3. Maintains randomness to explore a wide hyperparameter space.	1. Might discard potentially good combinations early if they perform poorly in initial limited training.
Use Case:	Ideal when working with large datasets or complex models where full Grid Search is too slow.

Bayesian Optimization

Definition: A smart **hyperparameter tuning method** used for **black-box optimization**, where the internal process of the model is unknown or too complex to analyze directly.

Basic Concept:

- It uses **past evaluation results** to **predict** which hyperparameter combinations are likely to perform best next.
- It builds a **probabilistic model** (often a Gaussian Process) of the objective function to guide the search efficiently.

Process:

1. Start with a few random hyperparameter combinations.
2. **Evaluate** their performance.
3. **Model the relationship** between hyperparameters and performance.
4. **Select the next set** of hyperparameters based on this learned relationship.
5. Repeat until an optimal or near-optimal set is found.

Advantages	Disadvantages
1. Much faster and more efficient than brute-force methods. 2. Reduces unnecessary evaluations by focusing on promising regions. 3. Ideal for computationally expensive models.	1. Complex to implement compared to Random or Grid Search. 2. Performance depends on how well the surrogate model approximates the objective function.
Use Case:	Best suited for expensive or high-dimensional models , such as deep learning or large-scale simulations.

Gradient-Based Optimization

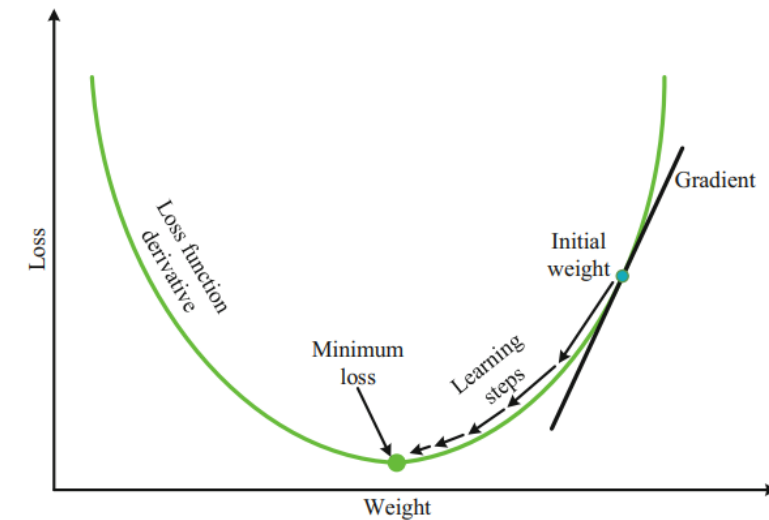
Recall: The main goal of training ML models is to **minimize the loss (cost) function** to achieve the **best set of parameters** (e.g., weights, biases).

Concept:

1. This optimization is done using gradient-based methods, which adjust parameters in the direction that reduces the loss.
2. The gradient represents the slope of the loss function; the model moves opposite to the gradient to reach the minimum.

Process (Gradient Descent Algorithm):

1. Compute the **gradient (derivative)** of the loss function with respect to model parameters.
2. **Update parameters** in the opposite direction of the gradient.
3. Repeat until the gradient is **close to zero** (minimum loss).



Gradient descent
visualization

Gradient-Based Optimization

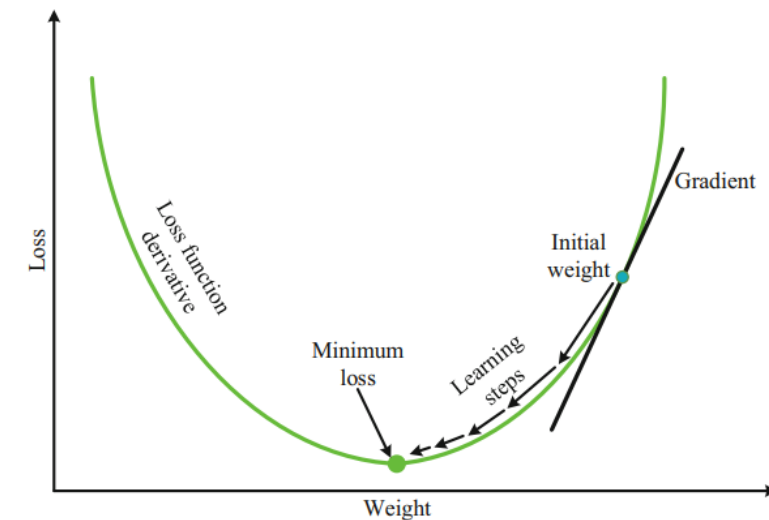
Types of Gradient Descent:

1. **Batch (Vanilla) Gradient Descent:** Uses the entire dataset for each update.
2. **Stochastic Gradient Descent (SGD):** Updates using one data point at a time.
3. **Mini-Batch Gradient Descent:** Updates using a small batch of data points (balance between speed and stability).

Learning Rate:

- A **hyperparameter** that controls **step size** in parameter updates.
 - If too high → may overshoot the minimum.
 - If too low → convergence becomes slow.

The **optimal parameters** correspond to the **minimum point of the loss curve**, where the derivative is zero.



Gradient descent
visualization

Evolutionary Optimization

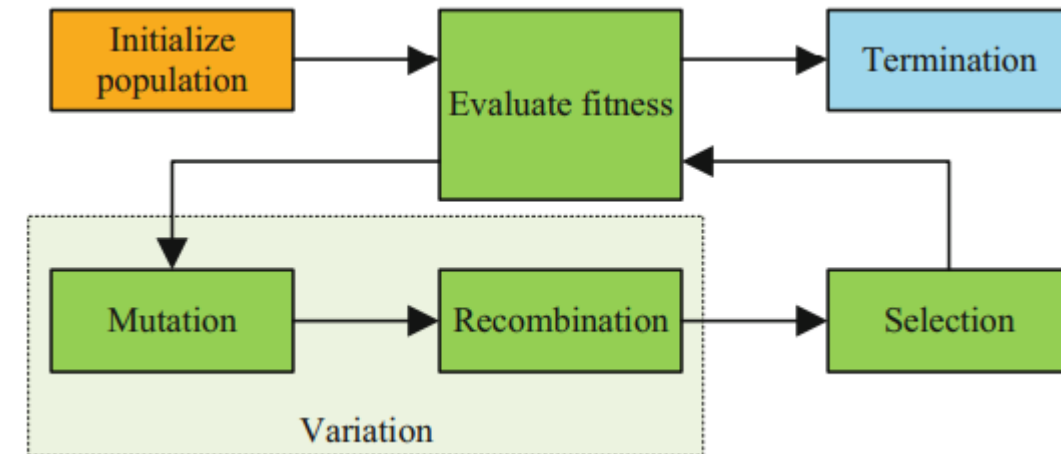
- An optimization technique inspired by biological evolution and Darwin's theory of natural selection.
- Used to find the **best hyperparameter sets** or model solutions through iterative improvement.

Concept:

- Treats each **solution (hyperparameter set)** as an **individual organism** in a **population**.
- The population evolves over time to improve overall performance.

Main Processes:

1. **Selection:** Choose the best-performing individuals (solutions).
2. **Reproduction:** Generate new solutions based on selected individuals.
3. **Mutation:** Randomly alter some parameters to introduce diversity.
4. **Recombination (Crossover):** Combine features from two or more individuals to create offspring.



Evolutionary algorithm
process that mimics the
biological process

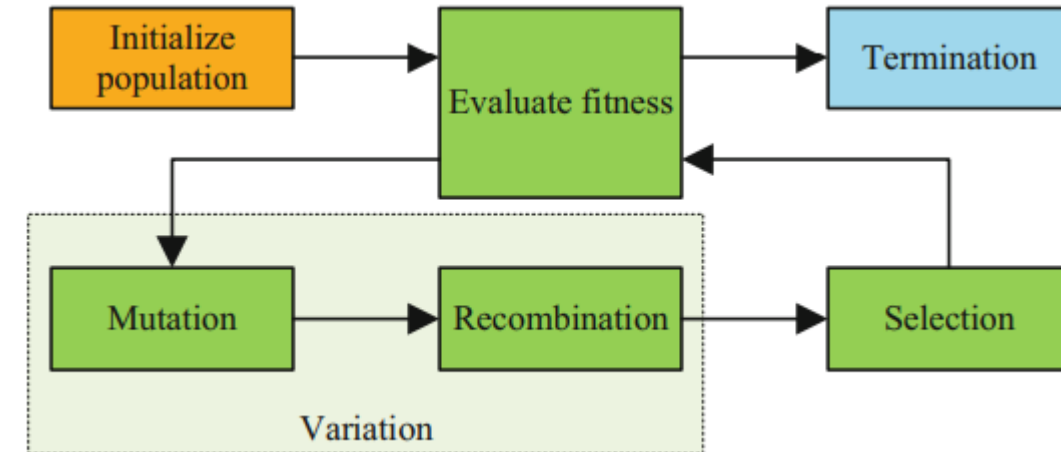
Evolutionary Optimization

Fitness Evaluation:

- Each solution's **fitness** is measured based on **model performance metrics**.
- Poor-performing individuals are **eliminated**, keeping only the fittest.

Mimics “**survival of the fittest**” — better solutions survive and evolve each generation.

Advantages	Disadvantages
- Effective for complex, non-linear, or multi-objective problems. - Can explore large search spaces without exhaustive computation.	Computationally expensive for large populations. - May require many generations to converge.



Evolutionary algorithm process that mimics the biological process

Test Your Knowledge

MCQ

The **Mean Squared Error (MSE)** is:

- A) Less sensitive to outliers
- ☒ B) More sensitive to outliers
- C) Used only for classification
- D) Based on absolute differences

Cross-validation helps in:

- ☒ A) Reducing bias and variance
- B) Increasing overfitting
- C) Decreasing training data
- D) Avoiding model evaluation

T / F

Cross-validation is less stable than a simple train-test split.

The learning rate is a hyperparameter.

Fill in blanks

The difference between predicted and actual values is called error

Stratified K-Fold maintains the same class ratios across folds.

THANK
you